



# Introduction to Linux system

Text streams, Pipes and Redirections

Prof. Badre BOUSALEM



# Agenda

- I. Processing text streams using filters.
- II. Use of text streams, pipes and redirections.
- III. Process management.
- IV. Management of execution priority

# **I. Processing text streams using filters.**





# Regular expressions(1/2)

Regular expressions are a series of characters used to make selections. They work with some commands like **grep** and **egrep**

**The main metacharacters of regular expressions are the following:**

|               |                                                                                                                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>.</b>      | Indicates any character                                                                                                                    |
| <b>-</b>      | Interval mark in a set                                                                                                                     |
| <b>*</b>      | Refers to any repetition of the preceding character                                                                                        |
| <b>^</b>      | Denotes the beginning of a line                                                                                                            |
| <b>\$</b>     | Indicates an end of line                                                                                                                   |
| <b>\</b>      | \ Protects the next character, whether it is a normal character or a metacharacter ( of regular expressions)                               |
| <b>{nbre}</b> | indicates the repetition number of the previous character, whether it is a normal character or a metacharacter (of regular expressions)    |
| <b>[...]</b>  | Denotes characters in brackets, defined by enumeration or by a range (^ after the [ denotes the complement of the characters in brackets). |

# Regular expressions(2/2)

Examples:

|                              |                                                                                                                                                  |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>'.*'</code>            | Denotes any line                                                                                                                                 |
| <code>'^\$'</code>           | Denotes an empty line                                                                                                                            |
| <code>'^début'</code>        | Denotes a line beginning with start                                                                                                              |
| <code>'\*.\ \'</code>        | Denotes a line containing a character *, followed by any character, followed by a character \                                                    |
| <code>'[A-Z][a-z]{9}'</code> | Denotes a line containing at least 10 successive letters, the first one being a capital letter, and the following nine being lower case letters. |
| <code>'^[A-Z]'</code>        | Indicates a line beginning with a capital letter                                                                                                 |

# grep command

To find a specific text in a file:

**grep [options] [string] [files]**

It is important to specify the search string so that the shell does not interpret it as a wildcard.

## Main options:

- i:** Ignore the case
- l:** Lists file names only if at least one matches.
- c:** Displays only the number of lines that match.
- n:** Also displays the line number.
- v:** Should not match.
- r:** Move recursively into directories.
- h:** Avoid adding prefixes to file names when multiple files are searched.

# grep command: Examples

→ Search "127" in the file "hosts"

```
grep "127" /etc/hosts
```

→ Count the occurrences of a word "Word" per file in the system log directory

```
grep -c "word" /var/log/*
```

→ Display all lines starting with the characters between a and d

```
grep ^[a-d] carnet-adresse
```

# egrep command

- **egrep** is an acronym for "Extended Global Regular Expressions Print".
- `grep` supports extended regular expressions or ERE using the option `-E`. This is similar to `egrep` command in Linux.
- The "E" in `egrep` refers to treating the pattern as a regular expression. "ERE" short for "Extended Regular Expressions" is enabled in `egrep`.
- `egrep` (which is the same as `grep -E`) treats `+`, `?`, `|`, `(`, and `)` as metacharacters.
- In basic regular expressions (with `grep`), the metacharacters `?`, `+`, `{`, `|`, (and `)` lose their special meaning. If you want to `grep` to treat these characters as metacharacters, escape them `\`, `\+`, `\{`, `\|`, `\(` (and `\)`).



# grep VS egrep

- grep basically supports basic regular expressions
- egrep supports extended regular expressions which offer more performance.
- The egrep command supports all the options of the grep command

# grep VS egrep: examples

- grep uses basic regular expressions in which the plus "+" is treated literally, all lines containing a plus "+" are returned.

```
grep "+" myfile.txt
```

- On the other hand, egrep considers the "+" as a metacharacter and returns every line because the sign "+" is interpreted as "one or more times".

```
egrep "+" myfile.txt
```

→ Each line is returned because the sign "+" was treated by egrep as a metacharacter. However, grep would have searched only for lines containing the sign "+".

## II. Use of text streams, pipes and redirections.



# Use of text streams and redirections:(pipes)

Tubes or pipes allow direct redirection of the output channel → of one command to → the input channel of another.

Example:

```
ls -l > resultat.txt
```

```
wc < resultat.txt
```

# Use of text streams, pipes and redirections :

Standard inputs and standard outputs (1/2)

For each executed command in a terminal, there is:

- A standard input value, 0 (the default keyboard on a console).
- A standard output value, 1 (By default the terminal).
- A standard output value for errors, 2 (by default the terminal).

Each channel can also be identified by an address:

- &0 for inputs,
- &1 for outputs,
- &2 for errors.

# Use of text streams, pipes and redirections :

Standard inputs and standard outputs (2/2)

Each channel [n] can be redirected:

- ❑ [n]< file: The default value of n is 0 and it reads standard input from the file.
- ❑ [n]> file: The default value of n is 1 and it sends the standard output to the file (clobber).
- ❑ [n]>>file: The default value is 1 and it appends the standard output to the file.
- ❑ <<word: Reads the following text until word is reached.
- ❑ `command` or \$(command): Replaces the command name with the output.



# Pipes

Pipes are the best way to apply several commands at the same time.

**command1 | command2**

It redirects the standard output of command1 to the standard input of command2.

**! We can't do standard error pipes.**

Example:

```
[root@test ] /# ls -l /dev | more
[root@test ] /# ls -l /etc/*.conf | grep user | grep 500
[root@test ] /# ls -l /bin | mail `users`
```

You have to use **tee** to duplicate the standard output in a file and on the standard output (by default terminal) at the same time.

```
[root@test ] /# ls -l /dev | tee file
[root@test ] /# ls -l /etc | tee -a file # Append to the file
```

# Construction of arguments

The **xargs** utility builds a list of arguments for a command using the standard inputs.

**xargs [options] [command]**

The **xargs** command creates a list of standard input arguments. It is used as a pipe.

Main option:

-p: offers a command prompt to the user before the execution of each command.

## Examples:

```
[root@test ] /# ls f* | xargs cat # Print to standard output the content of  
all files starting with f
```

```
[root@test ] /# find ~ -name 'proj1*' print | xargs cat # Search in the Home  
directory for files starting with proj1 and send it to the standard input of  
cat.
```

# The special file `/dev/null`

Use the `/dev/null` file for the suppression of error messages.

## Examples:

Try the following commands:

```
[root@test ] /# grep try /etc/*  
  
[root@test ] /# grep try /etc/* 2>/dev/null  
  
[root@test ] /# grep try /etc/*>/dev/null 2>/dev/null
```

### **III. Process Management.**



# Processes: Create, manage and kill a process (1/4)

An active application is a process.

Each process has:

- A process identifier (process ID).
- A parent process ID.
- A current PWD directory.
- A table of file descriptors (a file descriptor is the internal representation of a file opened by a process, it includes the mode in which it was opened).
- An executed program.
- Environment variables inherited from its parent process.
- Stdin, stdout, and stderr.
- Other.

Bash is a program and becomes a process when it is executed.

# Processes: Create, manage and kill a process (2/4)

Each time a command is executed in a Shell, a new process is created.

**An exception: builtin Shell commands. They are executed in the context of the Shell. You have to use the `type` command to check if a command is a builtin command.**

Example:

```
[root@test ] /# type cp ls which type
```

# Processes: Create, manage and kill a process (3/4)

Once the system is mounted and running from a terminal, it is possible to manage the processes started in the program with the command **ps**.

To display all the processes in the system, enter:

```
[root@test ] /# ps -Al
```

| F | S | UID  | PID  | PPID | C | PRI | NI | ADDR | SZ   | WCHAN  | TTY | TIME     | CMD         |
|---|---|------|------|------|---|-----|----|------|------|--------|-----|----------|-------------|
| 4 | S | 0    | 1    | 0    | 0 | 76  | 0  | -    | 469  | -      | ?   | 00:00:00 | init        |
| 1 | S | 0    | 2    | 1    | 0 | 94  | 19 | -    | 0    | ksofti | ?   | 00:00:00 | ksoftirqd/0 |
| 1 | R | 1000 | 4976 | 3574 | 0 | 76  | 0  | -    | 7287 | -      | ?   | 00:00:01 | konsole     |

The **ps** program will display all active processes, their PID and other information.

Classic **ps** options:

- A: select all processes
- l : long format
- a : select all processes, except session leaders and processes not attached to a terminal
- u [users] : select processes belonging to the user with the specified name or EUID
- x : select processes attached to a tty terminal (BSD option)



# Processes: Create, manage and kill a process (4/4)

To view all the processes in your session, enter:

```
[root@test ] /# ps -l
```

| F | S | UID | PID  | PPID | C | PRI | NI | ADDR | SZ  | WCHAN | TTY   | TIME     | CMD  |
|---|---|-----|------|------|---|-----|----|------|-----|-------|-------|----------|------|
| 4 | S | 0   | 5448 | 4981 | 0 | 77  | 0  | -    | 639 | wait  | pts/1 | 00:00:00 | su   |
| 0 | S | 0   | 5450 | 5448 | 0 | 75  | 0  | -    | 795 | wait  | pts/1 | 00:00:00 | bash |
| 0 | R | 0   | 7050 | 5450 | 0 | 76  | 0  | -    | 668 | -     | pts/1 | 00:00:00 | ps   |

F: Process Flags 002: being created, 040: forked but didn't exec, 400: killed by a signal

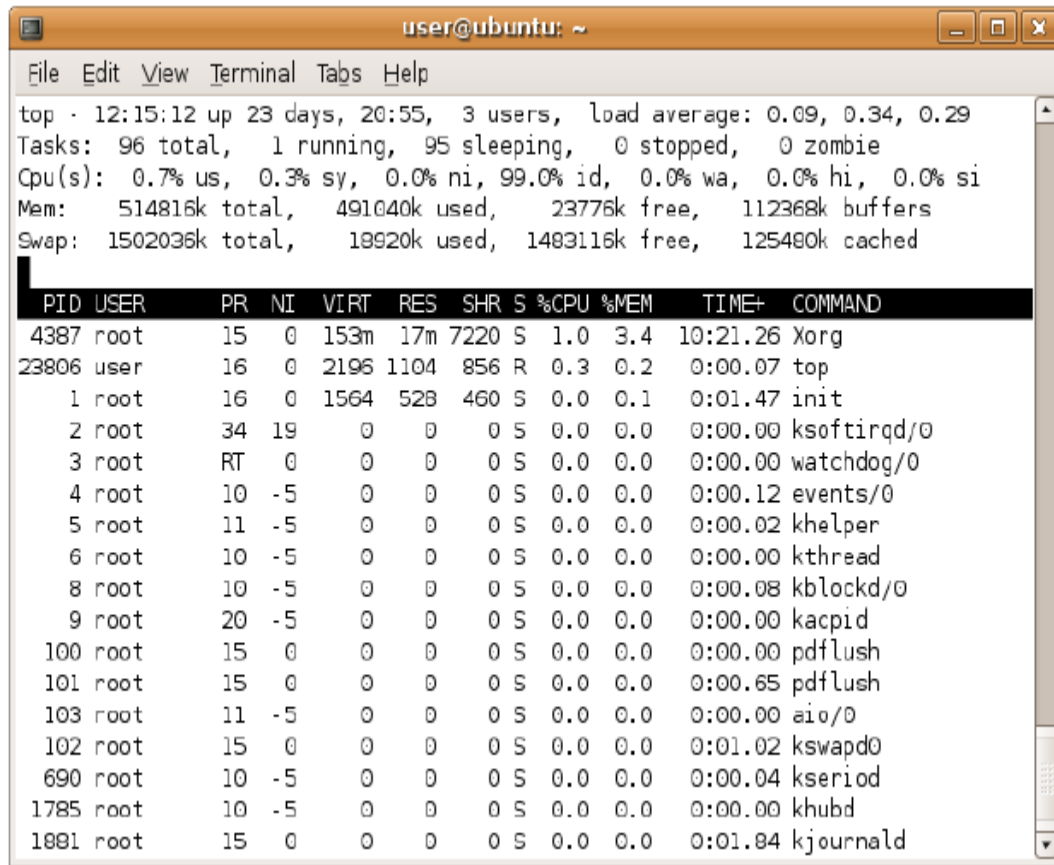
S: Process States: R: runnable, S: sleeping, Z: zombie.

UID: user ID, PID: Process ID, PPID: Parent Process ID, C: Scheduler, PRI: priority.

NI: nice value, SZ: size of routine, WCHAN: name of routine.

# Processes: Real-time process management (1/2)

To manage processes in real time, you must use **top**.



```
top - 12:15:12 up 23 days, 20:55, 3 users, load average: 0.09, 0.34, 0.29
Tasks: 96 total, 1 running, 95 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.7% us, 0.3% sy, 0.0% ni, 99.0% id, 0.0% wa, 0.0% hi, 0.0% si
Mem: 514816k total, 491040k used, 23776k free, 112368k buffers
Swap: 1502036k total, 18920k used, 1483116k free, 125480k cached
```

| PID   | USER | PR | NI | VIRT | RES  | SHR  | S | %CPU | %MEM | TIME+    | COMMAND     |
|-------|------|----|----|------|------|------|---|------|------|----------|-------------|
| 4387  | root | 15 | 0  | 153m | 17m  | 7220 | S | 1.0  | 3.4  | 10:21.26 | Xorg        |
| 23806 | user | 16 | 0  | 2196 | 1104 | 856  | R | 0.3  | 0.2  | 0:00.07  | top         |
| 1     | root | 16 | 0  | 1564 | 528  | 460  | S | 0.0  | 0.1  | 0:01.47  | init        |
| 2     | root | 34 | 19 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | ksoftirqd/0 |
| 3     | root | RT | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | watchdog/0  |
| 4     | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.12  | events/0    |
| 5     | root | 11 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.02  | khelper     |
| 6     | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | kthread     |
| 8     | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.08  | kblockd/0   |
| 9     | root | 20 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | kacpid      |
| 100   | root | 15 | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | pdflush     |
| 101   | root | 15 | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.65  | pdflush     |
| 103   | root | 11 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | aio/0       |
| 102   | root | 15 | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:01.02  | kswapd0     |
| 690   | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.04  | kseriod     |
| 1785  | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | khubd       |
| 1881  | root | 15 | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:01.84  | kjournald   |

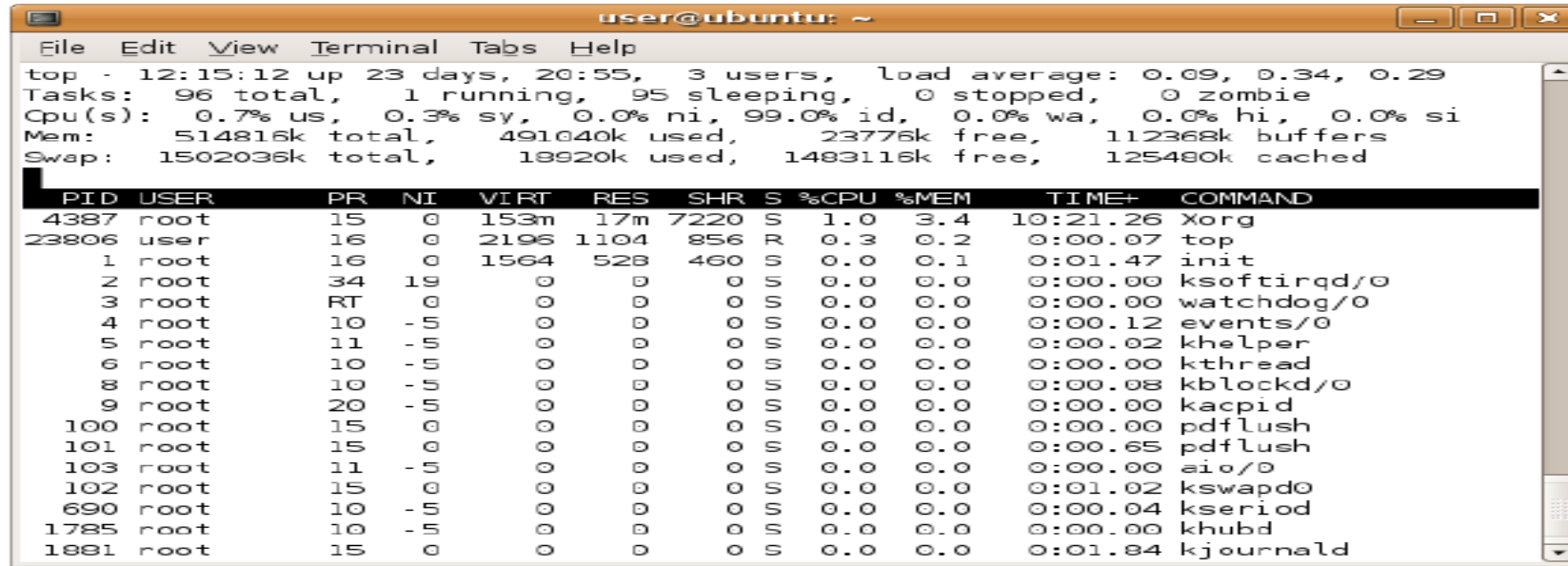
**RSS:** The total physical memory used by the task.

**SHARE:** The level of shared memory used by the task.

**%CPU:** The percentage of CPU time used by the task.

**%MEM:** The percentage of physical memory occupied by the task.

## Processes: Real-time process management (2/2)



top - 12:15:12 up 23 days, 20:55, 3 users, load average: 0.09, 0.34, 0.29  
Tasks: 96 total, 1 running, 95 sleeping, 0 stopped, 0 zombie  
Cpu(s): 0.7% us, 0.3% sy, 0.0% ni, 99.0% id, 0.0% wa, 0.0% hi, 0.0% si  
Mem: 514816k total, 491040k used, 23776k free, 112368k buffers  
Swap: 1502036k total, 18920k used, 1483116k free, 125480k cached

| PID   | USER | PR | NI | VIRT | RES  | SHR  | S | %CPU | %MEM | TIME+    | COMMAND     |
|-------|------|----|----|------|------|------|---|------|------|----------|-------------|
| 4387  | root | 15 | 0  | 153m | 17m  | 7220 | S | 1.0  | 3.4  | 10:21.26 | Xorg        |
| 23806 | user | 16 | 0  | 2196 | 1104 | 856  | R | 0.3  | 0.2  | 0:00.07  | top         |
| 1     | root | 16 | 0  | 1564 | 528  | 460  | S | 0.0  | 0.1  | 0:01.47  | init        |
| 2     | root | 34 | 19 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | ksoftirqd/0 |
| 3     | root | RT | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | watchdog/0  |
| 4     | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.12  | events/0    |
| 5     | root | 11 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.02  | khelper     |
| 6     | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | kthread     |
| 8     | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.08  | kblockd/0   |
| 9     | root | 20 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | kacpid      |
| 100   | root | 15 | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | pdflush     |
| 101   | root | 15 | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.65  | pdflush     |
| 103   | root | 11 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | aio/0       |
| 102   | root | 15 | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:01.02  | kswapd0     |
| 690   | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.04  | kseriod     |
| 1785  | root | 10 | -5 | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:00.00  | khubb       |
| 1881  | root | 15 | 0  | 0    | 0    | 0    | S | 0.0  | 0.0  | 0:01.84  | kjournald   |

Once top is launched, it is possible to execute interactive commands:

- Type N to sort the tasks by pid.
- Type A to sort the tasks in chronological order (most recent first).
- Type P to sort the tasks according to their CPU usage.
- Type M to sort tasks by memory usage.
- Type k to kill a process (prompted for pid).

# Processes: Send a signal to processes

- ✓ The ps program will display all active processes and their PID.
- ✓ If you know the PID, it is possible to send a signal to the process.
  - SIGSTOP (17) to stop a process.
  - SIGCONT (19) to restart a stopped process.
  - SIGTERM (15) to tell the process to kill itself.
  - SIGKILL (9) to kill a process.
- ✓ The program that allows to send a signal to the processes is: **kill**.

```
[root@test ] /# kill -SIGKILL [pid]  
[root@test ] /# kill -63 [pid]  
[root@test ] /# kill -l
```

- ✓ By default, a process is started in the foreground and is the only one to receive keyboard input.
- ✓ You must use CTRL+Z to suspend it.
- ✓ To start a process in the background, you must use the & sign.

# Processes: Job control

**&** : to start a job in the background

**Jobs**: to list all active jobs

**fg %1**: brought a job to the forefront.

To suspend a job: type Ctrl-Z (the job goes into a stopped state)

**bg %1**: to restart the execution of a suspended job

**kill %1**: kill a job

**!The number 1 followed by the character % corresponds to the first job executed.**

- When bash is terminated, all processes started from this shell receive the SIGHUP signal, which, by default, stops them.
- To prevent a process from stopping when its parent terminates, the program can be started with the command **nohup**:

```
user@ubuntu:~$ nohup xeyes
```

## **IV. Management of the execution priority.**





# Processes: the execution priorities of a process (1/2)

**nice**: To launch a command with an adjusted priority

```
nice -n -20 [command]
```

```
nice -n +19 [command]
```

- The program `nice` changes the size of the time frame assigned by the kernel scheduler. This value can be varied from -20 (highest priority) to +19 (lowest priority). The default value is 0. You must be root to assign a value between 1 and 20.



## Processes: the execution priorities of a process (2/2)

- You can change the priority of a running program by using the command **renice**.

```
renice [+|-] nicenumber [option] targets
```

This changes the nicenumber of one or more running processes to activate the priorities.

Main options:

**u**: Interprets targets as user names.

**p**: Default: Interprets targets as PID.

**g**: Forces parameters to be interpreted as a group process ID.  
group process.

Examples:

```
[root@test ] /# renice 20 501  
[root@test ] /# renice +1 987 -u daemon root -p 32
```